

Manual Técnico

TeacherBot

Plataforma de Gestión Académica
con Inteligencia Artificial

Unidad Educativa Oxford

Versión 1.0 — Agosto 2026

Documentación Técnica Interna

title: "Manual Técnico — TeacherBot"
subtitle: "Plataforma de Gestión Académica con Inteligencia Artificial"
author: "Unidad Educativa Oxford"
date: "Agosto 2026"
version: "1.0"
lang: "es"
toc: true
toc-depth: 3

Manual Técnico — TeacherBot

Plataforma de Gestión Académica con Inteligencia Artificial

Versión: 1.0
Fecha: Agosto 2026
Autor: Departamento de Sistemas — Unidad Educativa Oxford
Clasificación: Documentación Técnica Interna

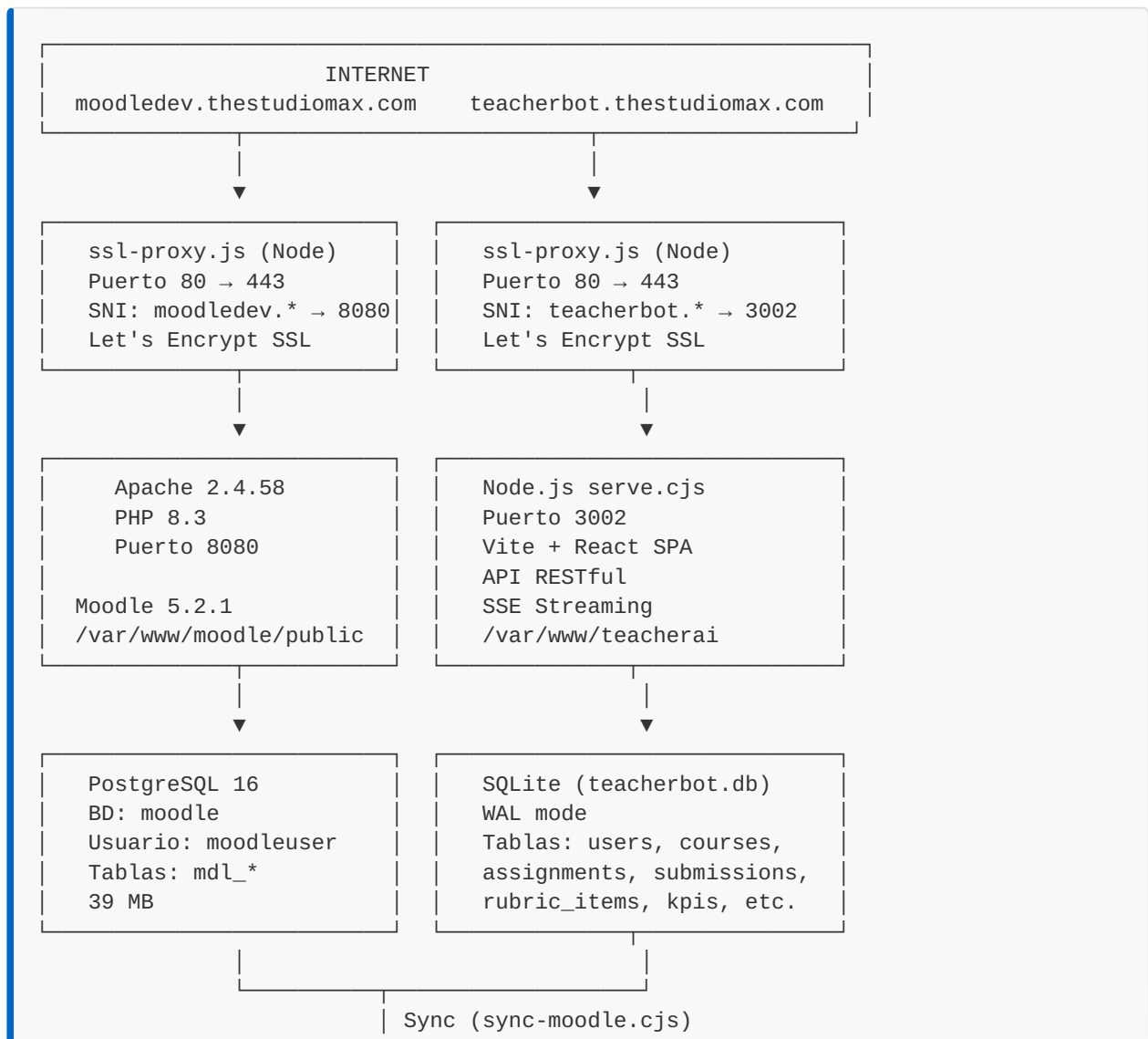
CAPÍTULO 1: Visión General de la Solución

1.1 Descripción del Sistema

TeacherBot es una plataforma de gestión académica diseñada para la **Unidad Educativa Oxford** que integra inteligencia artificial para la evaluación automatizada de tareas estudiantiles. El sistema se compone de dos aplicaciones principales que trabajan en conjunto:

1. **Moodle 5.2.1** — Sistema de Gestión de Aprendizaje (LMS) como backend académico
2. **TeacherBot** — Dashboard con IA para evaluación automatizada y monitoreo

1.2 Arquitectura General



▼

MiMo AI / OpenCode
Evaluación IA
deepseek-v4-pro
mimo-v2.5-pro

1.3 Stack Tecnológico

Infraestructura

Componente	Tecnología	Versión
Servidor	VPS Linux (Ubuntu 24.04)	6.8.0-106-generic
Proxy SSL	Node.js + http-proxy + Let's Encrypt	Personalizado
Base de Datos Moodle	PostgreSQL	16
Base de Datos TeacherBot	SQLite (better-sqlite3)	WAL Mode
Servidor Web Moodle	Apache + PHP	2.4.58 + 8.3
Servidor TeacherBot	Node.js (HTTP nativo)	22.x

Moodle

Componente	Detalle
Versión	5.2.1 (Build: 20260608)
Rama	502
Directorio	<code>/var/www/moodle/public</code>
Datos	<code>/var/www/moodledata</code>
Tema	Boost
Idioma	Español (es)
URL	<code>https://moodledev.thestudiomax.com</code>

TeacherBot

Componente	Tecnología	Versión
Frontend	Vite + React	18.3
UI Framework	shadcn/ui + Radix UI	Latest
Estilos	Tailwind CSS	3.4
Ruteo	React Router DOM	6.30
Estado	TanStack React Query	5.83
Gráficos	Recharts	2.15
Formularios	React Hook Form + Zod	7.61 + 3.25
PDF	jsPDF + jsPDF-AutoTable	4.0 + 5.0
Backend	Node.js HTTP Server	Monolítico
DB Driver	better-sqlite3	12.10
PostgreSQL Driver	pg	8.21

1.4 Módulos del Sistema

TeacherBot Dashboard

- **Panel Docente:** Gestión de cursos, tareas, estudiantes, evaluación manual e IA
- **Panel Vicerrector:** KPIs académicos, monitoreo de rendimiento, reportes
- **Panel Sistemas:** Monitoreo de infraestructura, logs, configuración de IA
- **Panel Admin:** Gestión de usuarios, proveedores IA, configuración global

API REST

El backend monolítico de Node.js expone las siguientes rutas:

Endpoint	Método	Descripción
/api/courses	GET	Lista cursos con tareas y entregas

Endpoint	Método	Descripción
<code>/api/assignments</code>	GET	Lista tareas con rúbricas
<code>/api/submissions</code>	GET	Lista entregas con calificaciones
<code>/api/students</code>	GET	Lista estudiantes
<code>/api/teachers</code>	GET	Lista docentes
<code>/api/stats</code>	GET	Estadísticas generales
<code>/api/kpis</code>	GET	KPIs del vicerrectorado
<code>/api/logs</code>	GET	Logs del sistema
<code>/api/health</code>	GET	Health check
<code>/api/sync</code>	POST	Dispara sincronización con Moodle
<code>/api/ai/evaluate</code>	POST	Evalúa una entrega con IA
<code>/api/ai/evaluate-stream/:id</code>	GET	Evalúa con streaming SSE en vivo
<code>/api/ai/evaluate-batch</code>	POST	Evalúa todas las entregas de una tarea
<code>/api/ai/status/:id</code>	GET	Estado de evaluación de una entrega
<code>/api/ai/config</code>	GET/PUT	Configuración de IA
<code>/api/ai/providers</code>	GET/POST	Gestión de proveedores IA
<code>/api/ai/providers/:id</code>	PUT/DELETE	CRUD de proveedor IA
<code>/api/ai/providers/:id/test</code>	POST	Test de conexión a proveedor
<code>/api/monitor/kpis</code>	GET	KPIs de infraestructura
<code>/api/monitor/tokens</code>	GET	Consumo de tokens (gráfico)
<code>/api/monitor/errors</code>	GET	Distribución de errores

1.5 Modelo de Datos

Tablas Principales (SQLite)

```
users
├── id (PK)
├── name
├── email
├── role: admin | vicerrector | docente | estudiante | sistemas
├── active
└── created_at

courses
├── id (PK)
├── name
├── period
├── level: primaria | secundaria | bachillerato
├── grade
├── subject
├── teacher_id (FK → users)
└── created_at

assignments
├── id (PK)
├── course_id (FK → courses)
├── title
├── prompt (instrucciones)
├── due_at
└── created_at

rubric_items
├── id (PK, autoincrement)
├── assignment_id (FK → assignments)
├── criterion
├── points
├── description
└── sort_order

students
├── id (PK)
├── name
├── email
├── grade
├── level
├── active
└── created_at

enrollments
├── id (PK, autoincrement)
├── student_id (FK → students)
├── course_id (FK → courses)
├── enrolled_at
└── UNIQUE(student_id, course_id)
```

```

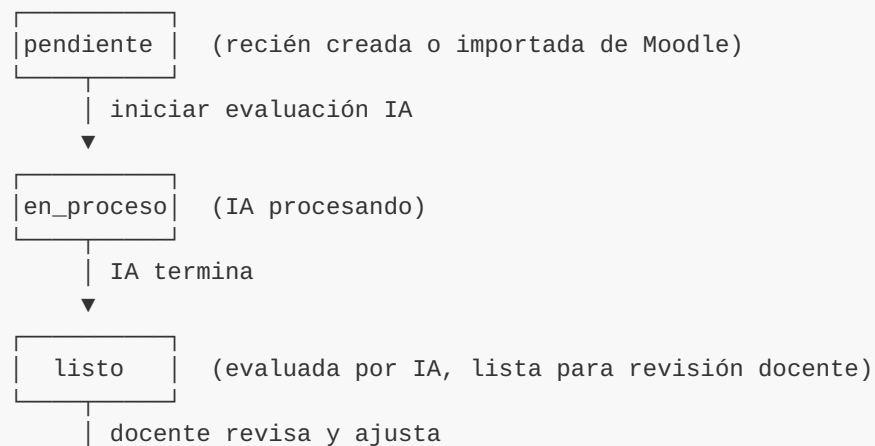
submissions
├── id (PK)
├── assignment_id (FK → assignments)
├── student_id (FK → students)
├── file_url
├── submitted_at
├── ai_score (0-10)
├── ai_feedback
├── teacher_score (0-10)
├── teacher_feedback
├── status: pendiente | en_proceso | listo | revisado
├── graded_at
└── graded_by (FK → users)

```

Tablas de Soporte

Tabla	Propósito
kpis	Indicadores de desempeño del vicerrectorado
kpi_details	Causas y efectos de cada KPI
system_logs	Logs del sistema (info, warn, error)
activity_log	Registro de actividad de usuarios
notifications	Notificaciones a usuarios
ai_config	Configuración del motor de IA (clave-valor)
ai_providers	Proveedores de IA configurados

1.6 Diagrama de Estados de Entrega





revisado

(calificación final confirmada)

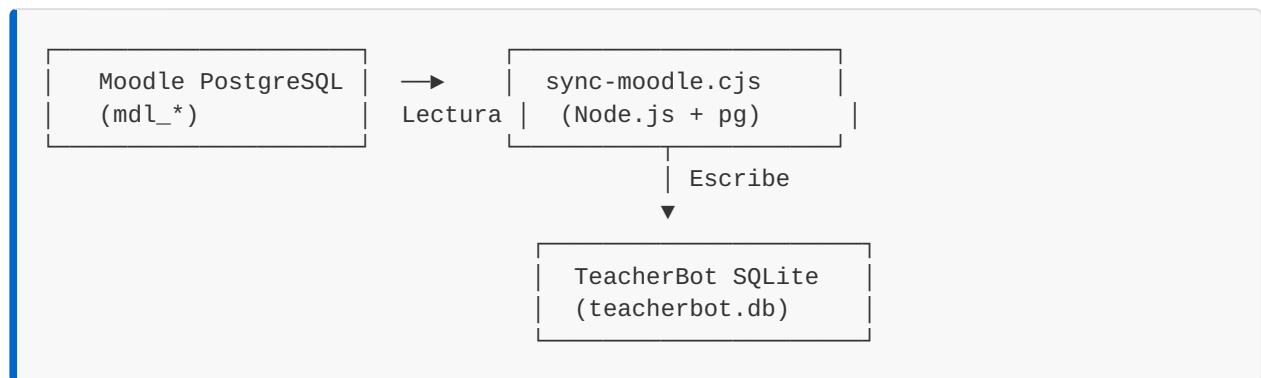
CAPÍTULO 2: Sincronización Moodle ↔ TeacherBot

2.1 Visión General

El motor de sincronización `sync-moodle.cjs` es el puente entre Moodle (fuente de verdad académica) y TeacherBot (plataforma de análisis y evaluación IA). Se ejecuta como un script Node.js independiente que:

1. Se conecta a la base de datos PostgreSQL de Moodle
2. Lee cursos, usuarios, tareas, entregas y calificaciones
3. Transforma y mapea los datos al esquema de TeacherBot (SQLite)
4. Copia archivos de entregas (PDFs) al sistema de archivos de TeacherBot
5. Genera rúbricas automáticas cuando no existen

2.2 Arquitectura de Sincronización



2.3 Configuración

La sincronización se configura mediante `moodle-config.json` :

```
{
  "moodle": {
    "version": "5.2.1",
    "url": "https://moodledev.thestudiomax.com",
    "db": {
      "type": "postgresql",
      "host": "127.0.0.1",
```

```

    "port": 5432,
    "user": "moodleuser",
    "password": "****",
    "database": "moodle",
    "prefix": "mdl_"
  },
  "courses": {
    "level": "bachillerato",
    "grade": "1ro BGU",
    "defaultPeriod": "Q2 2025-2026"
  }
},
"sync": {
  "importStudents": true,
  "importTeachers": true,
  "importCourses": true,
  "importAssignments": true,
  "importSubmissions": true,
  "importGrades": true,
  "mapGradesTo10Scale": true,
  "autoGenerateRubric": true
},
"mapping": {
  "roles": {
    "editingteacher": "docente",
    "teacher": "docente",
    "student": "estudiante",
    "manager": "admin"
  }
}
}
}

```

Parámetros de Sincronización

Parámetro	Tipo	Descripción
<code>importStudents</code>	boolean	Importar estudiantes desde Moodle
<code>importTeachers</code>	boolean	Importar docentes desde Moodle
<code>importCourses</code>	boolean	Importar cursos desde Moodle
<code>importAssignments</code>	boolean	Importar tareas desde Moodle
<code>importSubmissions</code>	boolean	Importar entregas desde Moodle
<code>importGrades</code>	boolean	Importar calificaciones desde Moodle
<code>mapGradesTo10Scale</code>	boolean	Convertir notas de 0-100 a 0-10
<code>autoGenerateRubric</code>	boolean	Generar rúbricas automáticas para tareas sin rúbrica

2.4 Flujo de Sincronización (6 Fases)

Fase 1: SyncUsers — Sincronización de Usuarios

```
SELECT u.id, u.username, u.firstname, u.lastname, u.email, r.shortname as role
FROM mdl_user u
JOIN mdl_role_assignments ra ON ra.userid = u.id
JOIN mdl_context ctx ON ra.contextid = ctx.id AND ctx.contextlevel = 50
JOIN mdl_role r ON ra.roleid = r.id
WHERE u.deleted = 0 AND u.confirmed = 1 AND u.id > 1
GROUP BY u.id, u.username, u.firstname, u.lastname, u.email, r.shortname
```

Mapeo de roles:

| Rol Moodle | Rol TeacherBot | Tablas |

|---|---|---|

| `editingteacher` , `teacher` | `docente` | `users` |

| `student` | `estudiante` | `users` + `students` |

| `manager` | `admin` | `users` |

Generación de IDs: `u-m-{moodle_id}` para docentes, `s-m-{moodle_id}` para estudiantes.

Fase 2: SyncCourses — Sincronización de Cursos

```
SELECT c.id, c.fullname, c.shortname, c.startdate,
       (SELECT ra.userid FROM mdl_role_assignments ra
        JOIN mdl_context ctx ON ra.contextid = ctx.id
        JOIN mdl_role r ON ra.roleid = r.id
        WHERE ctx.instanceid = c.id AND ctx.contextlevel = 50
        AND r.shortname = 'editingteacher' LIMIT 1) as teacher_moodle_id
FROM mdl_course c
WHERE c.id != 1
```

Generación de IDs: `c-m-{moodle_id}`

Fase 3: SyncEnrollments — Sincronización de Matrículas

```
SELECT DISTINCT ra.userid, ctx.instanceid as course_id
FROM mdl_role_assignments ra
JOIN mdl_context ctx ON ra.contextid = ctx.id AND ctx.contextlevel = 50
JOIN mdl_role r ON ra.roleid = r.id
WHERE r.shortname = 'student' AND ctx.instanceid != 1
```

Relaciona estudiantes con cursos en la tabla `enrollments`.

Fase 4: SyncAssignments — Sincronización de Tareas

```
SELECT a.id, a.course, a.name, a.intro, a.duedate, a.grade
FROM mdl_assign a
```

Generación de IDs: `a-m-{moodle_id}`

Generación automática de rúbricas: Cuando `autoGenerateRubric = true`, se crea una rúbrica de 4 criterios basada en los puntos totales de la tarea:

Criterio	Peso
Contenido y Precisión	40%
Estructura y Organización	25%
Creatividad y Originalidad	20%
Presentación y Ortografía	15%

Fase 5: SyncSubmissions — Sincronización de Entregas y Calificaciones

Consulta de entregas:

```
SELECT s.id, s.assignment, s.userid, s.status, s.timemodified
FROM mdl_assign_submission s
WHERE s.status = 'submitted'
ORDER BY s.id
```

Consulta de calificaciones:

```
SELECT g.id, g.assignment, g.userid, g.grader, g.grade, g.timemodified
FROM mdl_assign_grades g
```

Consulta de retroalimentación:

```
SELECT fc.assignment, fc.grade as grade_id, fc.commenttext
FROM mdl_assignfeedback_comments fc
WHERE fc.commenttext IS NOT NULL AND fc.commenttext != ''
```

Mapeo de estados:

| Estado Moodle | Estado TeacherBot |

|---|---|

| `draft` | `pendiente` |

```
| submitted | pendiente |
| graded   | listo   |
| reopened | en_proceso |
```

Conversión de notas: Las notas de Moodle (escala 0-100) se convierten a la escala 0-10 de TeacherBot:

```
nota_teacherbot = Math.round((nota_moodle / 100) * 10 * 10) / 10
```

Fase 6: SyncFiles — Sincronización de Archivos

```
SELECT s.id as sub_id, f.filename, f.contenthash
FROM mdl_assign_submission s
JOIN mdl_files f ON f.itemid = s.id
  AND f.component = 'assignsubmission_file'
  AND f.filearea = 'submission_files'
  AND f.filename != '.'
WHERE s.status = 'submitted'
```

Los archivos se copian desde el sistema de archivos de Moodle (`/var/www/moodledata/filedir/{hash}`) al directorio de datos de TeacherBot (`/var/www/teacherai/data/`), con el nombre `moodle_sub_{id}_{filename}` .

2.5 Ejecución

Manual

```
node /var/www/teacherai/sync-moodle.cjs --config /var/www/teacherai/moodle-
config.json
```

Vía API

```
curl -X POST https://teacherbot.thestudiomax.com/api/sync
```

Salida esperada

```
TeacherBot ← Moodle Sync Engine
```

```
Moodle 5.2.1 → 127.0.0.1/moodle
```

```

* TeacherBot DB: /var/www/teacherai/teacherbot.db
  PostgreSQL conectado

* Sincronizando usuarios...
  15 usuarios sincronizados

* Sincronizando cursos...
  5 cursos sincronizados

* Sincronizando matrículas...
  12 matrículas sincronizadas

* Sincronizando tareas...
  7 tareas sincronizadas

Sincronizando entregas y calificaciones...
  21 entregas sincronizadas (15 con calificación)

Sincronizando archivos de entregas...
  8 archivos importados de Moodle

```

TeacherBot Database Summary

users	15 registros
courses	5 registros
assignments	7 registros
rubric_items	28 registros
students	12 registros
enrollments	12 registros
submissions	21 registros

Sincronización completada.

2.6 Mapeo de Identificadores

Para mantener la trazabilidad, TeacherBot utiliza un sistema de prefijos que vinculan cada registro con su origen en Moodle:

Entidad	Prefijo TeacherBot	Ejemplo
Docente	u-m-{id}	u-m-42
Estudiante	s-m-{id}	s-m-157
Curso	c-m-{id}	c-m-5
Tarea	a-m-{id}	a-m-23

Entidad	Prefijo TeacherBot	Ejemplo
Entrega	sub-m-{id}	sub-m-891

Esta convención permite identificar inmediatamente si un registro proviene de Moodle y facilita la auditoría y resolución de conflictos.

CAPÍTULO 3: Inteligencia Artificial y Proceso de Calificación

3.1 Visión General

TeacherBot integra un motor de evaluación con inteligencia artificial que analiza las entregas de los estudiantes contra rúbricas predefinidas y genera:

- **Nota numérica** (escala 0-10)
- **Feedback por criterio** de la rúbrica
- **Fortalezas** identificadas
- **Áreas de mejora**
- **Recomendación final** accionable

El sistema soporta **múltiples proveedores de IA** configurable en caliente, con un sistema de fallback automático.

3.2 Proveedores de IA

TeacherBot utiliza una arquitectura multi-proveedor que permite conmutar entre diferentes servicios de IA:

Proveedor	Modelo	Estado	Prioridad
OpenCode Go	deepseek-v4-pro	Activo (Default)	1
MiMo (Xiaomi)	mimo-v2.5-pro	Activo	2
OpenAI	gpt-4o-mini	Deshabilitado	3

Configuración de Proveedor

Cada proveedor se define con los siguientes campos:

```
{
  "id": "prov-1781056672344",
  "name": "OpenCode Go",
  "provider_type": "openai",
  "api_url": "https://opencode.ai/zen/go/v1/chat/completions",
```

```
"api_key": "sk-9...RJ75",
"model": "deepseek-v4-pro",
"is_default": true,
"enabled": true
}
```

API de Gestión de Proveedores

Endpoint	Descripción
GET /api/ai/providers	Listar todos los proveedores
POST /api/ai/providers	Crear nuevo proveedor
PUT /api/ai/providers/:id	Actualizar proveedor
DELETE /api/ai/providers/:id	Eliminar proveedor
POST /api/ai/providers/:id/test	Probar conexión

3.3 Configuración del Motor de IA

La configuración se almacena en la tabla `ai_config` (clave-valor):

Clave	Valor	Descripción
<code>enable_ai_evaluation</code>	<code>true</code>	Activar/desactivar evaluación IA
<code>model</code>	<code>deepseek-v4-pro</code>	Modelo por defecto
<code>temperature</code>	<code>0.5</code>	Creatividad de la IA (0-1)
<code>max_tokens</code>	<code>8000</code>	Tokens máximos por respuesta
<code>evaluation_language</code>	<code>es</code>	Idioma de evaluación
<code>auto_approve_threshold</code>	<code>7.0</code>	Umbral de aprobación automática
<code>enable_cache</code>	<code>true</code>	Cachear resultados de evaluación
<code>batch_delay_ms</code>	<code>2000</code>	Delay entre evaluaciones en lote
<code>rubric_weight_content</code>	<code>40</code>	Peso del contenido en rúbrica auto

Clave	Valor	Descripción
<code>rubric_weight_structure</code>	25	Peso de estructura
<code>rubric_weight_creativity</code>	20	Peso de creatividad
<code>rubric_weight_presentation</code>	15	Peso de presentación

3.4 Flujo de Evaluación con IA

El proceso de evaluación sigue 9 pasos secuenciales, transmitidos en tiempo real vía **Server-Sent Events (SSE)**:

```

PASO 1 → inicio
        Inicializa el motor de evaluación IA

PASO 2 → cargando_entrega
        Consulta la base de datos: entrega, rúbrica y datos del estudiante
        SELECT s.*, st.name, a.title, a.prompt, a.course_id
        FROM submissions s
        JOIN students st ON s.student_id = st.id
        JOIN assignments a ON s.assignment_id = a.id
        WHERE s.id = ?

PASO 3 → entrega_encontrada
        Confirma carga exitosa: estudiante + tarea

PASO 4 → rubrica_cargada
        Carga los criterios de evaluación desde rubric_items
        SELECT * FROM rubric_items
        WHERE assignment_id = ?
        ORDER BY sort_order

PASO 5 → extrayendo_texto / texto_extraido
        Si hay PDF: extrae texto con pdftotext
        Si no hay PDF: genera contexto de evaluación simulado

PASO 6 → enviando_ia
        Construye el prompt y lo envía al proveedor IA

PASO 7 → respuesta_recibida
        Recibe y parsea la respuesta JSON de la IA

PASO 8 → criterio_evaluado (× N criterios)
        Emite resultado por cada criterio de la rúbrica

PASO 9 → nota_calculada + completado
        Nota final y feedback consolidado

```

3.5 Diseño del Prompt de Evaluación

System Prompt (Rol del Evaluador)

Eres un profesor experto evaluando tareas académicas.
Tu trabajo es analizar la entrega de un estudiante contra una rúbrica y dar feedback constructivo.

Debes responder EXACTAMENTE en este formato JSON
(nada más, sin markdown alrededor):

```
{
  "nota": <número 0-10>,
  "feedback_general": "<2-4 oraciones con resumen general>",
  "criterios": [
    {
      "criterio": "<nombre exacto del criterio>",
      "puntos_max": <número>,
      "puntos_obtenidos": <número>,
      "comentario": "<1-2 oraciones>"
    }
  ],
  "fortalezas": ["<fortaleza 1>", "<fortaleza 2>", "<fortaleza 3>"],
  "areas_mejora": ["<área de mejora 1>", "<área de mejora 2>"],
  "recomendacion_final": "<1 oración con consejo accionable>"
}
```

La nota debe calcularse así:
(suma de puntos_obtenidos / {maxPoints}) * 10.
Sé justo pero exigente.

User Message (Contexto de Evaluación)

```
## Instrucciones de la Tarea
{prompt de la tarea}

## Rúbrica (Total: {maxPoints} puntos)
1. **Criterio 1** (X pts): Descripción
2. **Criterio 2** (Y pts): Descripción
...

## Entrega del Estudiante: {nombre}
```

{contenido del PDF o texto extraído (máx 8000 caracteres)}

Evalúa esta entrega según la rúbrica. Responde SOLO con el JSON.

3.6 Procesamiento de la Respuesta

Parseo JSON

La respuesta de la IA se limpia de bloques markdown (``json ``) y se parsea como JSON:

```
const clean = result
  .replace(/```json\n?/g, '')
  .replace(/```\n?/g, '')
  .trim();
const evaluation = JSON.parse(clean);
```

Cálculo de Nota

```
const aiScore = Math.min(10, Math.max(0, Number(evaluation.nota) || 0));
```

Actualización en Base de Datos

La evaluación se guarda en la tabla `submissions`:

```
db.prepare(`
  UPDATE submissions
  SET ai_score = ?, ai_feedback = ?, status = 'listo', graded_at = datetime('now')
  WHERE id = ?
`).run(aiScore, fullFeedback, submissionId);
```

3.7 Sistema de Fallback

Cuando el proveedor de IA principal falla (timeout, error de red, rate limit), el sistema activa un mecanismo de respaldo que genera una **evaluación simulada**:

1. Se detecta el error del proveedor IA
2. Se notifica al frontend: " IA no disponible — generando estimación automática de respaldo"
3. Se calculan puntuaciones aleatorias dentro del rango 60%-95% de los puntos máximos por criterio
4. Se genera feedback genérico indicando que es una estimación
5. Se marca la entrega para reintento posterior

```
// Simulación de puntuaciones por criterio
const obtained = Math.max(1,
```

```
Math.round(r.points * (0.6 + Math.random() * 0.35) * 10) / 10
);
```

3.8 Streaming en Tiempo Real (SSE)

La evaluación puede seguirse en vivo mediante Server-Sent Events:

```
GET /api/ai/evaluate-stream/{submissionId}?provider={providerId}
```

Eventos emitidos:

```
event: progress
data: {"step":"inicio","message":"Iniciando motor de evaluación
IA...","ts":1691000000000}

event: progress
data: {"step":"cargando_entrega","message":"Consultando base de datos..."}

event: progress
data: {"step":"rubrica_cargada","message":"Rúbrica cargada: 4 criterios · Total:
10 puntos"}

event: progress
data: {"step":"enviando_ia","message":"Enviando a OpenCode Go para análisis..."}

event: progress
data: {"step":"criterio_evaluado","message":"Contenido y Precisión: 8.5/10 pts
(85%)"}

event: progress
data: {"step":"nota_calculada","message":"Nota final calculada: 8.3/10", "detail":
{"score":8.3}}

event: complete
data: {"id":"sub-001", "ai_score":8.3, "ai_feedback":"..."}

event: error
data: {"message":"Timeout MiMo"}
```

3.9 Evaluación por Lotes

Para evaluar todas las entregas pendientes de una tarea:

```
POST /api/ai/evaluate-batch
{
```

```
"assignment_id": "a1"
}
```

El sistema:

1. Consulta todas las entregas pendientes (`status = 'pendiente' OR 'en_proceso'`)
2. Evalúa cada una secuencialmente con un delay de 2 segundos entre evaluaciones
3. Retorna un resumen con éxitos y errores

```
{
  "evaluated": 12,
  "errors": 1,
  "results": [
    {"id": "sub-001", "status": "success", "score": 8.7},
    {"id": "sub-002", "status": "error", "error": "Timeout MiMo"}
  ]
}
```

El delay de 2 segundos (`batch_delay_ms = 2000`) previene rate limiting de los proveedores IA.

3.10 Sistema de Caché

Para evitar evaluaciones duplicadas y reducir costos de API:

```
const existing = db.prepare(
  'SELECT ai_score, ai_feedback, status FROM submissions WHERE id = ?'
).get(submission_id);

if (existing && existing.ai_score !== null && existing.ai_feedback) {
  console.log(`  ${submission_id}: Ya evaluado (${existing.ai_score}/10),
devolviendo cache`);
  return json(res, {
    id: submission_id,
    ai_score: existing.ai_score,
    ai_feedback: existing.ai_feedback,
    status: existing.status,
    from_cache: true
  });
}
```

Una entrega ya evaluada retorna el resultado cacheado instantáneamente sin consumir tokens de IA.

3.11 Monitoreo de Consumo de Tokens

El dashboard de sistemas monitorea el consumo de tokens:

Métrica	Valor Actual
Tokens usados (hoy)	80,000
Límite diario	200,000
Porcentaje usado	40%
Entregas evaluadas	20
Pendientes	1
Última evaluación	10 Jun 2026

Estimación de tokens por evaluación:

- Prompt del sistema: ~300 tokens
- Prompt del usuario (rúbrica + entrega): ~2,000-4,000 tokens
- Respuesta de la IA: ~500-1,000 tokens
- **Total estimado por evaluación: ~4,000 tokens**

3.12 Seguridad y Manejo de Errores

Protección de API Keys

Las claves de API se enmascaran en todas las respuestas:

```
if (p.api_key && p.api_key.length > 8) {
  p.api_key_masked = p.api_key.substring(0, 4) + '...' +
    p.api_key.substring(p.api_key.length - 4);
}
```

Timeout y Reintentos

- Timeout de conexión: 180 segundos (3 minutos)
- Timeout de test: 15 segundos
- Sin reintentos automáticos (el usuario decide reintentar)

Manejo de Errores

```
Error detectado en callAI()
```



```

¿Error de red / timeout?
  → Generar evaluación simulada de respaldo
  → Marcar status = 'pendiente' (reintentar)

¿JSON inválido de la IA?
  → Error: "La IA devolvió formato inválido"
  → No se guarda resultado

¿Respuesta vacía?
  → Error: "Respuesta vacía del proveedor"
  → No se guarda resultado

```

3.13 Ejemplo Completo de Evaluación

Entrada

- **Estudiante:** Ana García
- **Tarea:** Ensayo — Argumentación
- **Rúbrica:**
 - Tesis y coherencia (3 pts)
 - Evidencia y citas (3 pts)
 - Estructura y redacción (2 pts)
 - Ortografía y estilo (2 pts)
- **Total:** 10 puntos

Prompt Enviado a la IA

```

## Instrucciones de la Tarea
Evalúa el ensayo según la rúbrica. Entrega: (1) nota 0-10,
(2) feedback por criterio, (3) 3 recomendaciones accionables.
Mantén tono respetuoso y académico.

## Rúbrica (Total: 10 puntos)
1. **Tesis y coherencia** (3 pts): La tesis es clara y se sostiene
   con coherencia en todo el texto.
2. **Evidencia y citas** (3 pts): Usa evidencias pertinentes y
   referencias/citas adecuadas.
3. **Estructura y redacción** (2 pts): Introducción, desarrollo y
   conclusión bien articulados; redacción correcta.
4. **Ortografía y estilo** (2 pts): Ortografía, puntuación y estilo
   apropiados para el nivel.

## Entrega del Estudiante: Ana García
[Contenido del ensayo...]

```

Respuesta de la IA

```
{
  "nota": 8.7,
  "feedback_general": "Excelente ensayo con una tesis clara y bien defendida a lo largo del texto. La estructura es sólida y la argumentación coherente. Se recomienda reforzar el uso de citas formales y revisar algunos detalles de puntuación en la conclusión.",
  "criterios": [
    {
      "criterio": "Tesis y coherencia",
      "puntos_max": 3,
      "puntos_obtenidos": 2.8,
      "comentario": "Tesis bien definida y mantenida consistentemente. La línea argumental es clara."
    },
    {
      "criterio": "Evidencia y citas",
      "puntos_max": 3,
      "puntos_obtenidos": 2.4,
      "comentario": "Buen uso de evidencias, pero faltan citas formales en dos párrafos clave."
    },
    {
      "criterio": "Estructura y redacción",
      "puntos_max": 2,
      "puntos_obtenidos": 1.9,
      "comentario": "Estructura bien organizada con introducción, desarrollo y conclusión claros."
    },
    {
      "criterio": "Ortografía y estilo",
      "puntos_max": 2,
      "puntos_obtenidos": 1.6,
      "comentario": "Buena ortografía general. Revisar signos de puntuación en la conclusión."
    }
  ],
  "fortalezas": [
    "Tesis clara y bien articulada",
    "Excelente organización estructural",
    "Argumentación coherente y fluida"
  ],
  "areas_mejora": [
    "Incorporar citas formales con formato académico",
    "Revisar puntuación en párrafos finales"
  ],
  "recomendacion_final": "Para la próxima entrega, incluye referencias bibliográficas en formato APA y revisa la puntuación de los párrafos conclusivos para elevar la calidad académica del texto."
}
```

Resultado Final en TeacherBot

Nota IA: 8.7/10

Evaluación por Criterios

- Tesis y coherencia: 2.8/3 pts – Tesis bien definida...
- Evidencia y citas: 2.4/3 pts – Buen uso de evidencias...
- Estructura y redacción: 1.9/2 pts – Estructura bien organizada...
- Ortografía y estilo: 1.6/2 pts – Buena ortografía general...

Fortalezas

- Tesis clara y bien articulada
- Excelente organización estructural
- Argumentación coherente y fluida

Áreas de Mejora

- Incorporar citas formales con formato académico
- Revisar puntuación en párrafos finales

Recomendación Final

Para la próxima entrega, incluye referencias bibliográficas en formato APA y revisa la puntuación de los párrafos conclusivos para elevar la calidad académica del texto.

APÉNDICE A: Comandos de Operación

Inicio del Sistema

```
# Moodle (Apache)
systemctl start apache2

# TeacherBot
systemctl start teacherai

# SSL Proxy
systemctl start ssl-proxy
```

Sincronización Manual

```
node /var/www/teacherai/sync-moodle.cjs
# o vía API:
curl -X POST https://teacherbot.thestudiomax.com/api/sync
```

Verificar Estado

```
systemctl status apache2 teacherai ssl-proxy
curl -s https://teacherbot.thestudiomax.com/api/health
curl -s https://moodledev.thestudiomax.com/login/index.php | head -5
```

Logs

```
journalctl -u teacherai -f
journalctl -u apache2 -f
tail -f /var/log/apache2/moodle_error.log
```

APÉNDICE B: Puertos y Rutas de Red

Servicio	Puerto	Proceso
SSL Proxy HTTP	80	ssl-proxy.js (Node)
SSL Proxy HTTPS	443	ssl-proxy.js (Node)
TeacherBot	3002	serve.cjs (Node)
Moodle (Apache)	8080	apache2
PostgreSQL	5432	postgresql

Dominio	Proxy →	Servicio
<code>agentiko.thestudiomax.com</code>	→ :3000	SIGEX
<code>moodledev.thestudiomax.com</code>	→ :8080	Moodle
<code>teacherbot.thestudiomax.com</code>	→ :3002	TeacherBot
<code>delivery.thestudiomax.com</code>	→ :3001	DeliveryApp

APÉNDICE C: Glosario

Término	Definición
TeacherBot	Plataforma de gestión académica con IA para evaluación automatizada
Moodle	LMS (Learning Management System) de código abierto
SSE	Server-Sent Events — streaming unidireccional servidor → cliente
Rúbrica	Conjunto de criterios de evaluación con puntuaciones
Prompt Engineering	Diseño de instrucciones para modelos de IA
WAL	Write-Ahead Logging — modo de journaling de SQLite
MiMo	Modelo de IA de Xiaomi (mimo-v2.5-pro)
SNI	Server Name Indication — enrutamiento TLS por nombre de dominio

Documento generado el 3 de agosto de 2026 — Departamento de Sistemas, Unidad Educativa Oxford